

GenerationConfig

https://huggingface.co/docs/transformers/main/en/main_classes/text_generation

Detailed Documentation

`class`

`transformers.GenerationConfig`

`class`

`transformers.GenerationConfig` https://github.com/huggingface/transformers/blob/main/src/transformers/generation/configuration_utils.py

`Parameters that control the length of the output`

- `max_length` (`int`, *optional*, defaults to 20) --

The maximum length the generated tokens can have. Corresponds to the length of the input prompt +

`max_new_tokens`. Its effect is overridden by `max_new_tokens`, if also set.

- `max_new_tokens` (`int`, *optional*) --

The maximum numbers of tokens to generate, ignoring the number of tokens in the prompt.

- `min_length` (`int`, *optional*, defaults to 0) --

The minimum length of the sequence to be generated. Corresponds to the length of the input prompt +

`min_new_tokens`. Its effect is overridden by `min_new_tokens`, if also set.

- `**min_new_tokens**` (`int`, *optional*) --

`from_pretrainedtransformers.GenerationConfig.from_pretrainedh`

```
from_pretrainedtransformers.GenerationConfig.from_pretrainedhttps://github.com
"pretrained_model_name", "val": ": typing.Union[str, os.PathLike]"}, {"name":
"config_file_name", "val": ": typing.Union[str, os.PathLike, NoneType] = None"},
{"name": "cache_dir", "val": ": typing.Union[str, os.PathLike, NoneType] =
None"}, {"name": "force_download", "val": ": bool = False"}, {"name":
"local_files_only", "val": ": bool = False"}, {"name": "token", "val": ":
typing.Union[bool, str, NoneType] = None"}, {"name": "revision", "val": ": str =
'main'"}, {"name": "***kwargs", "val": ""}]- **pretrained_model_name** ( str or
os.PathLike ) --
```

This can be either:

- a string, the *model id* of a pretrained model configuration hosted inside a model repo on
- a path to a *directory* containing a configuration file saved using the

```
[ save_pretrained ()]
```

(/docs/transformers/main/en/main_classes/text_generation# `transformers.Gener`
method, e.g., `./my_model_directory/` .

- `**config_file_name**` (`str` or `os.PathLike`, *optional*, defaults to `"generation_config.json"`) --

Name of the generation configuration JSON file to be loaded from `pretrained_model_name`.

- `**cache_dir**` (`str` or `os.PathLike`, *optional*) --

Path to a directory in which a downloaded pretrained model configuration should be cached if the

standard cache should not be used.

from_model_configtransformers.GenerationConfig.from_model_c

from_model_configtransformers.GenerationConfig.from_model_confighttps://github.com/huggingface/transformers/blob/main/src/transformers/configuration_utils.py#L1000-L1010

```
"model_config", "val": "PreTrainedConfig"]- **model_config**
( PreTrainedConfig ) --
```

The model config that will be used to instantiate the generation config.
0[GenerationConfig]

(/docs/transformers/main/en/main_classes/text_generation#transformers.GenerationConfig) --
configuration object instantiated from those parameters.

Instantiates a [GenerationConfig]

(/docs/transformers/main/en/main_classes/text_generation#transformers.GenerationConfig) --

from a [PreTrainedConfig]

(/docs/transformers/main/en/main_classes/configuration#transformers.PreTrainedConfig) --

This function is useful to convert legacy

[PreTrainedConfig]

(/docs/transformers/main/en/main_classes/configuration# transformers.PreTrai

objects, which may contain generation parameters, into a stand-alone
[GenerationConfig]

(/docs/transformers/main/en/main_classes/text_generation# transformers.Gener

save_pretrainedtransformers.GenerationConfig.save_pretrainedh

save_pretrainedtransformers.GenerationConfig.save_pretrainedhttps://github.com

```
"save_directory", "val": ": typing.Union[str, os.PathLike]"}, {"name":  
"config_file_name", "val": ": typing.Union[str, os.PathLike, NoneType] = None"},  
{"name": "push_to_hub", "val": ": bool = False"}, {"name": "**kwargs", "val": ""}]-  
**save_directory** ( str or os.PathLike ) --
```

Directory where the configuration JSON file will be saved (will be created if it
does not exist).

- **config_file_name** (str or os.PathLike , *optional*, defaults to
"generation_config.json") --

Name of the generation configuration JSON file to be saved in
save_directory .

- **push_to_hub** (bool , *optional*, defaults to False) --

Whether or not to push your model to the Hugging Face model hub after saving
it. You can specify the

repository you want to push to with repo_id (will default to the name of

`save_directory` in your

- `**kwargs` (`dict[str, Any]`, *optional*) --

Additional key word arguments passed along to the [`push_to_hub` ()] (/docs/transformers/main/en/main_classes/model#transformers.utils.PushToHubMixin.push_to_hub) method.

Save a generation configuration object to the directory `save_directory`, so that it can be re-loaded using the

`updatetransformers.GenerationConfig.updatehttps://github.com/`

`updatetransformers.GenerationConfig.updatehttps://github.com/huggingface/transformers`
`***kwargs", "val": ""}] - **kwargs** ( dict[str, Any] ) --`

Dictionary of attributes to tentatively update this class.0 `dict[str, Any]` Dictionary containing all the key-value pairs that were not used to update the instance.

Updates attributes of this class instance with attributes from `kwargs` if they match existing attributes,

returning all the unused kwargs.

`validatetransformers.GenerationConfig.validatehttps://github.com/`

`validatetransformers.GenerationConfig.validatehttps://github.com/huggingface/transformers`

"strict", "val": " = False"]}- **strict** (bool) -- If True, raise an exception for any issues found. If False, only log issues.0

Validates the values of the attributes of the [GenerationConfig] (/docs/transformers/main/en/main_classes/text_generation# `transformers.Gener` instance. Raises exceptions in the presence

of parameterization that can be detected as incorrect from the configuration instance alone.

Note that some parameters not validated here are best validated at generate runtime, as they may depend on

other inputs and/or the model, such as parameters related to the generation length.

get_generation_modetransformers.GenerationConfig.get_generat

get_generation_modetransformers.GenerationConfig.get_generation_modehttps://
"assistant_model", "val": ": typing.Optional[ForwardRef('PreTrainedModel')] =
None"]}- **assistant_model** (`PreTrainedModel`, *optional*) --

The assistant model to be used for assisted generation. If set, the generation mode will be

assisted generation.0 `GenerationMode` The generation mode triggered by the instance.

Returns the generation mode triggered by the [GenerationConfig]

(/docs/transformers/main/en/main_classes/text_generation# `transformers.Generation` instance.

`class` `<span class="code-`

`inline">transformers.GenerationMixin``transformers.GenerationMix`

`class`

`transformers.GenerationMixin``transformers.GenerationMixin``https`://github.

A class containing all functions for auto-regressive text generation, to be used as a mixin in model classes.

Inheriting from this class causes the model to have special generation-related behavior, such as loading a

`GenerationConfig` at initialization time or ensuring `generate`-related tests are run in `transformers` CI.

A model class should inherit from `GenerationMixin` to enable calling methods like `generate`, or when it

has defined a custom `generate` method that relies on `GenerationMixin`, directly or indirectly, which

approximately shares the same interface to public methods like `generate`.

Three examples:

- `LlamaForCausalLM` should inherit from `GenerationMixin` to enable calling `generate` and other public

methods in the mixin;

- `BlipForQuestionAnswering` has a custom `generate` method that approximately shares the same interface as

`transformers.GenerationMixin.generate` <https://github.com>

```
transformers.GenerationMixin.generate
"""
    Generate text for the given model and configuration.

    Args:
        inputs: torch.Tensor of shape (batch_size, sequence_length) or (batch_size, sequence_length, embedding_dim)
        generation_config: transformers.GenerationConfig object
        max_length: int, maximum length of the generated text
        min_length: int, minimum length of the generated text
        top_k: int, number of highest probability vocabulary tokens to keep for topk-filtering
        top_p: float, probability mass for nucleus sampling, beta in the equation p_beta_j = P(j in S | prefix) ** beta
        do_sample: bool, whether to sample from the output distribution of the model (i.e. stochastic mode)
        num_return_sequences: int, number of sequences to generate
        early_stopping: bool, whether to stop the generation when the model has generated a token that is not in the vocabulary
        pad_token_id: int, padding token id
        eos_token_id: int, end of sentence token id
        output_attentions: bool, whether to return the attentions during generation
        output_hidden_states: bool, whether to return the hidden states during generation
        return_dict: bool, whether to return a dictionary instead of a tuple
        kwargs: additional keyword arguments to pass to the model

    Returns:
        torch.Tensor of shape (batch_size, sequence_length) or (batch_size, sequence_length, embedding_dim)
"""
def generate(
    self,
    inputs: typing.Optional[torch.Tensor] = None,
    generation_config: typing.Optional[transformers.generation.configuration_utils.GenerationConfig] = None,
    max_length: int = 20,
    min_length: int = 0,
    top_k: int = 50,
    top_p: float = 1.0,
    do_sample: bool = False,
    num_return_sequences: int = 1,
    early_stopping: bool = False,
    pad_token_id: Optional[int] = None,
    eos_token_id: Optional[int] = None,
    output_attentions: bool = False,
    output_hidden_states: bool = False,
    return_dict: bool = False,
    **kwargs,
) --
    """
    Generate text for the given model and configuration.

    Args:
        inputs: torch.Tensor of shape (batch_size, sequence_length) or (batch_size, sequence_length, embedding_dim)
        generation_config: transformers.GenerationConfig object
        max_length: int, maximum length of the generated text
        min_length: int, minimum length of the generated text
        top_k: int, number of highest probability vocabulary tokens to keep for topk-filtering
        top_p: float, probability mass for nucleus sampling, beta in the equation p_beta_j = P(j in S | prefix) ** beta
        do_sample: bool, whether to sample from the output distribution of the model (i.e. stochastic mode)
        num_return_sequences: int, number of sequences to generate
        early_stopping: bool, whether to stop the generation when the model has generated a token that is not in the vocabulary
        pad_token_id: int, padding token id
        eos_token_id: int, end of sentence token id
        output_attentions: bool, whether to return the attentions during generation
        output_hidden_states: bool, whether to return the hidden states during generation
        return_dict: bool, whether to return a dictionary instead of a tuple
        kwargs: additional keyword arguments to pass to the model

    Returns:
        torch.Tensor of shape (batch_size, sequence_length) or (batch_size, sequence_length, embedding_dim)
    """
    # Set generation config if not provided
    if generation_config is None:
        generation_config = self.generation_config

    # Set min_length to max_length if not provided
    if min_length is None:
        min_length = max_length

    # Set top_k to 50 if not provided
    if top_k is None:
        top_k = 50

    # Set top_p to 1.0 if not provided
    if top_p is None:
        top_p = 1.0

    # Set do_sample to False if not provided
    if do_sample is None:
        do_sample = False

    # Set num_return_sequences to 1 if not provided
    if num_return_sequences is None:
        num_return_sequences = 1

    # Set early_stopping to False if not provided
    if early_stopping is None:
        early_stopping = False

    # Set pad_token_id to None if not provided
    if pad_token_id is None:
        pad_token_id = generation_config.pad_token_id

    # Set eos_token_id to None if not provided
    if eos_token_id is None:
        eos_token_id = generation_config.eos_token_id

    # Set output_attentions to False if not provided
    if output_attentions is None:
        output_attentions = False

    # Set output_hidden_states to False if not provided
    if output_hidden_states is None:
        output_hidden_states = False

    # Set return_dict to False if not provided
    if return_dict is None:
        return_dict = False

    # Generate text
    outputs = self.generate(
        inputs=inputs,
        max_length=max_length,
        min_length=min_length,
        top_k=top_k,
        top_p=top_p,
        do_sample=do_sample,
        num_return_sequences=num_return_sequences,
        early_stopping=early_stopping,
        pad_token_id=pad_token_id,
        eos_token_id=eos_token_id,
        output_attentions=output_attentions,
        output_hidden_states=output_hidden_states,
        return_dict=return_dict,
        **kwargs,
    )

    # Return the generated text
    return outputs
```


The sequence used as a prompt for the generation or as model inputs to the encoder. If `None` the

method initializes it with `bos_token_id` and a batch size of 1. For decoder-only models `inputs`

should be in the format of `input_ids`. For encoder-decoder models `*inputs*` can represent any of

`input_ids`, `input_values`, `input_features`, or `pixel_values`.

- `**generation_config**` ([GenerationConfig] (/docs/transformers/main/en/main_classes/text_generation# `transformers.G` *optional*) --

The generation configuration to be used as base parametrization for the generation call. `**kwargs`

passed to generate matching the attributes of `generation_config` will override them. If

`generation_config` is not provided, the default will be used, which has the following loading

priority: 1) from the `generation_config.json` model file, if it exists; 2) from the model

`compute_transition_score` `transformers.GenerationMixin.compute`

```
compute_transition_score transformers.GenerationMixin.compute_transition_score
"sequences", "val": ": Tensor"}, {"name": "scores", "val": ": tuple"}, {"name":
"beam_indices", "val": ": typing.Optional[torch.Tensor] = None"}, {"name":
"normalize_logits", "val": ": bool = False}]- **sequences** ( torch.LongTensor )
--
```

The generated sequences. The second `dimension` (sequence_length) is either equal to `max_length` or

shorter if all batches finished early due to the `eos_token_id`.

- `**scores**` (tuple (torch.FloatTensor)) --

Transition scores for each vocabulary token at each generation step. Beam transition scores consisting

of l